

AI CAREER PREPARATION AGENT

Complete Architecture & Build Handbook • Beginner-to-Production Guide

Version: v2.4.0 (Enterprise Academic Edition)	Target Audience: Complete Beginners to Full-Stack Engineers
Frontend: React 18, Vite, Tailwind CSS v4, Web Audio	Backend & AI: FastAPI, SQLite/PostgreSQL, Google Gemini 1.5/2.0

Chapter 1: Executive Overview & Product Vision

The AI Career Preparation Agent is an end-to-end, enterprise-grade pedagogical platform designed to empower job candidates to excel in high-stakes technical, behavioral, and architectural interviews. Unlike generic mock interview tools that ask static, repetitive questions, this platform functions as an autonomous pedagogical coach.

It synthesizes dynamic questions tailored to the candidate's chosen target career role, transcribes and evaluates answers in real-time across 5 rigorous rubric axes (Technical Knowledge, Relevance, STAR Communication, Problem-Solving Structure, and Delivery Confidence), and closes the preparation loop by measuring real competency gains over time.

■ **Core Pedagogical Philosophy:** No candidate should ever see fabricated baseline scores. Data integrity is absolute: zero-evidence fresh accounts start at an uncalibrated 0% baseline, while real scores are earned through verified practice drills, mock interviews, and ATS resume audits.

Chapter 2: High-Level System Architecture Blueprint

The system employs a unified, resilient client-server architecture. In development, the React 18 Single Page Application (SPA) runs via Vite with hot module replacement (HMR), while FastAPI serves REST APIs via Uvicorn. In production, a multi-stage Docker container compiles the frontend into static assets ('dist/') and FastAPI mounts and serves these assets directly, providing a lightning-fast single-port container deployment.

System Architectural Layers:

- 1. Presentation Layer: React 18 SPA, Tailwind CSS v4, Lucide Icons, MediaRecorder & SpeechRecognition APIs.
- 2. State & Context Layer: ThemeContext, AuthContext, InterviewContext with multi-tier storage synchronization.
- 3. API Gateway & Routing: FastAPI RESTful endpoints with automatic OpenAPI documentation and CORS protection.
- 4. AI & Inference Orchestrator: Multi-tier LLM factory prioritizing Google Gemini API (1.5/2.0 Flash) -> OpenAI -> Local Grounded Heuristic Engine.
- 5. RAG & Knowledge Retrieval: In-memory TF-IDF and semantic chunk retrieval grounded in role playbooks.
- 6. Question Vault & Community Exchange: Crowdsourced real company interview questions with automated PII scrubbing.
- 7. Persistence Layer: SQLAlchemy 2.0 ORM with SQLite database (portable development) or PostgreSQL (cloud production).

■ **Resilient Multi-Tier Fallback:** If an external LLM key is absent or exhausts its rate limit, the system gracefully degrades to the Local Grounded Engine instead of crashing. Candidates experience uninterrupted interview sessions with high-fidelity rubric feedback.

Chapter 3: Technology Stack Encyclopedia (The Why, Where, & How)

Every single library, framework, and tool in this repository was selected deliberately to maximize performance, maintainability, and zero-crash resilience. The matrix below details the exact justification for each component:

Technology	Role & Location	Why Chosen (Rationale)	How It Operates
React 18	Frontend Framework src/	Declarative component tree, concurrency, widespread industry standard.	Manages virtual DOM, state hooks, and reactive interview lifecycles.
Vite v8	Build Tool & Dev Server vite.config.js	Sub-second dev server startup, lightning production bundling with Rolldown.	Compiles 1,900+ modules into minified JS/CSS chunks in ~600ms.
Tailwind CSS v4	Design System & Styling src/index.css	Modern CSS variable bindings (@theme), zero-runtime overhead.	Supplies utility classes and drives dynamic theme color tokens.
FastAPI	Backend Web API backend/app/main.py	Python 3 async support, native Pydantic typing, auto Swagger UI docs.	Handles REST routes, authentication, evaluations, and background schedulers.
Google Gemini API	Primary LLM Engine gemini_service.py	State-of-the-art reasoning, ultra-fast TTFT, generous context window.	Synthesizes adaptive questions and grades answers via HTTPS REST protocol.
OpenAI Adapter	Secondary LLM Engine llm_service.py	Ensures interoperability with OpenAI, Azure, Groq, or DeepSeek.	Emits OpenAI chat completion requests if OPENAI_API_KEY is supplied.
Local Grounded LLM	Failsafe Fallback llm_service.py	Guarantees 100% uptime: never crashes even if offline or quota exceeded.	Applies deterministic rubric heuristics and playbook embeddings.
SQLAlchemy 2.0	Database ORM backend/app/db/	Robust relational mapping, prevents SQL injection, database agnostic.	Manages schemas, relationships, and queries across SQLite and PostgreSQL.
MediaRecorder API	Browser Telemetry MockInterviewPage.jsx	Native browser video/audio capture without heavy external npm plugins.	Streams webcam and microphone buffers into standard WebM/MP4 containers.
Web Speech API	Live Voice Transcription MockInterviewPage.jsx	Zero-latency real-time candidate verbal transcription in-browser.	Converts spoken speech to text chunks as the candidate speaks.
APScheduler	Background Cron reminder_service.py	Autonomous background task scheduling in pure Python.	Dispatches proactive daily practice reminder alerts based on user timezone.
Docker	Containerization Dockerfile	Consistent builds across Windows, macOS, Linux, and Cloud instances.	Multi-stage build compiling Vite frontend and serving via Uvicorn.

Chapter 4: Complete Codebase Directory Tour & File Manifest

A clear directory structure is essential for beginners navigating the project. The tree below highlights the purpose of each key folder and file across both frontend and backend:

Path / Directory	Primary Responsibility	Key Files Included
src/components/	Reusable presentation UI components	GlassCard.jsx, GradientButton.jsx, Badge.jsx, ProgressBar.jsx
src/components/video/	Multimodal video interview tools	RecordingControls.jsx, AiInterviewerAvatar.jsx, RecordingReview.jsx
src/context/	Global application state stores	AuthContext.jsx, InterviewContext.jsx, ThemeContext.jsx
src/pages/	Application route view controllers	DashboardPage.jsx, MockInterviewPage.jsx, SkillGapPage.jsx, SettingsPage.jsx
src/theme/	Theme metadata & color palettes	themeConfig.js (System, Warm, White, Dark theme presets)

Path / Directory	Primary Responsibility	Key Files Included
src/utlis/	Client-side calculation engines	skillGapAnalyzer.js, activityService.js, feedbackAgent.js
src/utlis/storage/	Client storage persistence facade	storageService.js (Scoped localStorage management)
backend/app/api/	FastAPI endpoint route definitions	routes/interview.py, profile.py, experiences.py, skills.py
backend/app/db/	Database configuration & models	database.py, models.py (Users, Interviews, Questions, Skills)
backend/app/schemas/	Pydantic request & response contracts	interview.py, profile.py, experience.py, activity.py
backend/app/services/	Core business logic & engine services	interview_engine.py, skill_service.py, experience_service.py
backend/app/services/ai/	AI, RAG, and LLM implementations	llm/gemini_service.py, llm/llm_service.py, rag/retrieval_service.py
tests/	Automated backend unit & integration tests	test_real_llm_and_role_sync.py, test_phase17_experiences.py

Chapter 5: Core Engines & Key Features Breakdown

5.1 Multimodal Mock Interview Engine

The mock interview engine orchestrates realistic technical sessions. Candidates select their Target Role, Interview Focus (Technical, Behavioral STAR, HR, Role Deep Dive), Difficulty (Beginner, Intermediate, Advanced), and Mode (Text, Video, Face-to-Face Voice). The interview engine executes dynamic 180-second countdowns, records continuous audio/video telemetry, transcribes verbal answers in real-time, and branches adaptively.

5.2 5-Axis Rubric Answer Evaluation System

Candidate answers are graded objectively against five standardized dimensions:

- 1. Technical Accuracy (0-100): Validates technical concepts, algorithms, frameworks, and syntax.
- 2. Relevance to Question (0-100): Checks if the candidate directly answered the core inquiry.
- 3. Communication & STAR Method (0-100): Measures structured storytelling (Situation, Task, Action, Result).
- 4. Problem Solving & Architecture (0-100): Assesses system tradeoffs, scale, and algorithmic efficiency.
- 5. Delivery Confidence & Conciseness (0-100): Evaluates clarity, authoritative tone, and avoidance of filler words.

5.3 Question Vault & Real Company Experience Integration

The Question Vault (/interview-experiences) is a crowdsourced repository where candidates contribute authentic questions and interview experiences from companies like Stripe, Google, Amazon, and startups. Key integration highlight: Questions submitted and approved in the Question Vault are automatically retrieved by backend/app/services/interview_engine.py via get_vault_questions_for_role(). The engine weaves these questions directly into mock interview sessions for matching target roles, badging them in the UI as: '■■ Community Question Vault • Real Company Question'.

5.4 Deterministic Gamification & Habit Loop

Preparation consistency is driven by an immutable activity engine (src/utlis/activityService.js). Every completed interview awards +100 XP (with bonuses up to +25 XP for high scores), daily drills award +50 XP, and resume scans award +35 XP. The engine calculates consecutive daily streaks deterministically from timestamped activities. Fresh accounts start at 0d streak and 0 XP without hardcoded mock fallbacks.

5.5 Four-Theme Architectural Visual Engine

- The application offers four distinct visual presentation themes:
- 1. System Preference: Dynamically responds to the candidate's operating system dark/light mode.
 - 2. Warm Scholar: Classical academic dossier with archival parchment (#F8F6F0), porcelain surfaces, and navy accents.
 - 3. Pure White: Modern, crisp executive white canvas (FFFFFF) with slate ink and cobalt blue highlights.
 - 4. Midnight Dark: Deep obsidian canvas (#0B0F17) with midnight surfaces, white typography, and electric cyan accents.

5.6 Custom Target Role Customization

In addition to the 6 preset career roles, candidates can define any custom target role (e.g. 'Site Reliability Engineer', 'NLP Research Scientist', 'Fintech Product Manager'). The custom role persists to user storage and the database, dynamically steering LLM question synthesis and competency analytics.

Chapter 6: Database Schemas & Data Models

The relational database schema is structured to ensure complete user isolation, data integrity, and strict foreign key cascading. Below is the relational entity overview:

Table Name	Primary Key	Foreign Keys	Key Fields & Responsibilities
users	id (VARCHAR)	None	email, name, role, target_role, xp, level, streak, created_at
profiles	id (VARCHAR)	user_id -> users.id	target_role, experience_years, skills_json, feedback_language
interviews	id (VARCHAR)	user_id -> users.id	role, interview_type, difficulty, overall_score, status, rubric_scores_json
interview_questions	id (VARCHAR)	session_id -> interviews.id	sequence_number, question, skill, difficulty, generated_source
interview_answers	id (VARCHAR)	question_id -> questions.id	answer_text, duration_seconds, audio_url, created_at
interview_experiences	id (VARCHAR)	user_id -> users.id	company, role, experience_text, difficulty, outcome, moderation_status
interview_experience_questions	id (VARCHAR)	experience_id -> experiences.id	question_text, round_type, topic, difficulty
activities	id (VARCHAR)	user_id -> users.id	activity_type, title, xp_earned, timestamp

Chapter 7: REST API Endpoints Specification

The backend exposes RESTful endpoints with automatic Swagger OpenAPI documentation at ``docs``:

HTTP Method	Endpoint Route	Purpose	Auth Required
POST	/api/auth/register	Register new user account	No
POST	/api/auth/login	Authenticate and return JWT token	No
GET / PUT	/api/profile	Retrieve and update user career profile	Yes
POST	/api/interview/start	Initiate a mock session (synthesizes Q1)	Yes
POST	/api/interview/answer	Submit candidate answer & grade rubric	Yes
GET	/api/interview/{id}	Fetch interview results & score matrix	Yes

HTTP Method	Endpoint Route	Purpose	Auth Required
GET / POST	/api/experiences	Browse & submit Question Vault entries	Yes
POST	/api/resume/analyze	Upload resume and perform ATS gap audit	Yes
GET	/api/analytics/competencies	Fetch real competency growth matrix	Yes

Chapter 8: Step-by-Step Beginner Build Guide

If you are starting from scratch and want to build this complete application from ground zero, follow these exact 7 steps:

Step	Action	Command / Execution Details
1	Install Prerequisites	Install Node.js (v18 or v20 LTS), Python (v3.11+), and Git.
2	Clone Repository	<code>git clone https://github.com/chandolupraneethkumar05-oss/AI-Career-Preparation-Agent.git</code> <code>cd AI-Career-Preparation-Agent</code>
3	Setup Backend	<code>cd backend</code> <code>python -m venv venv</code> # Windows: <code>venv\Scripts\activate</code> Mac/Linux: <code>source venv/bin/activate</code> <code>pip install -r requirements.txt</code>
4	Configure .env	Create .env with: PORT=8000 GEMINI_API_KEY=your_gemini_api_key GEMINI_MODEL=gemini-1.5-flash DATABASE_URL=sqlite:///./interview_ai.db
5	Setup Frontend	In project root: <code>npm install</code> <code>npm run build</code> # Verifies Vite bundling and CSS compilation
6	Run Dev Servers	Terminal 1 (Backend): <code>uvicorn backend.app.main:app --reload --port 8000</code> Terminal 2 (Frontend): <code>npm run dev</code> (Access <code>http://localhost:5173</code>)
7	Execute Tests	Run: <code>python -m pytest</code> Expected: 41 passed out of 41 tests (100% pass)

Chapter 9: Production Cloud Deployment Manual (Docker & Render)

Deploying to production requires zero manual server configuration using the multi-stage Docker build pipeline.

Multi-stage Dockerfile Explained:

- Stage 1 (Frontend Builder): Uses `node:20-slim`, runs `npm install`, and executes `npm run build` to output the optimized single-page bundle into `dist/` in ~800ms.
- Stage 2 (Backend Runtime): Uses `python:3.11-slim`, installs system build utilities (gcc, g++, curl), installs Python packages from `backend/requirements.txt`, copies backend application code, and copies static `dist/` assets from Stage 1 into the container.
- Single-Port Serving: In production, FastAPI mounts `dist/` at the root path (`/`) while mounting API routes under `/api`. Uvicorn serves both web clients and backend endpoints on port 10000.

- **One-Click Render.com Deployment Steps:**
1. Push code to your GitHub repo on branch 'main'.
 2. In Render.com dashboard, click 'New +' -> 'Web Service' and select your repository.
 3. Select 'Docker' as the Environment runtime.
 4. In 'Environment Variables', set GEMINI_API_KEY to your Google AI Studio API key.
 5. Click 'Deploy Web Service'. Render will build the Docker image, run database migrations, and provide a live HTTPS URL in ~3 minutes!